

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

LLMaJ Checks Reference

LLM-as-Judge (LLMaJ) checks use GPT-5.5 to evaluate task quality and correctness. These go beyond syntax to assess whether your task is well-designed.

Running LLMaJ Locally

BASH

 Copy

```
# GPT-5.5 (matches CI)
harbor tasks check -m openai/@openai/gpt-5.5 -o output.json harbor_
```

LLMaJ Checks

behavior_in_task_description

What it checks: Whether all behavior checked in tests is described in the `instruction.md` file.

Why it matters: Agents can only do what they're told. If tests check for behavior not mentioned in instructions, the task is unfair.

How to fix:

1. Review each test case
2. Ensure the corresponding behavior is explicitly stated in `instruction.md`
3. Add any missing requirements to instructions

Example:

PYTHON

 Copy

```
# Test checks for header row
def test_csv_has_header():
    with open("/output/data.csv") as f:
        first_line = f.readline()
        assert "id,name,value" in first_line
```

MARKDOWN

 Copy

```
# instruction.md must mention this!

Output a CSV file with headers: id, name, value
```

behavior_in_tests

What it checks: Whether all behavior described in `instruction.md` is verified by unit tests.

Why it matters: If instructions say to do something but no test checks it, the task has gaps.

How to fix:

1. Review each requirement in `instruction.md`
2. Ensure there's a corresponding test
3. Add tests for any uncovered requirements

informative_test_docstrings

What it checks: Whether test functions have docstrings explaining what they check.

Why it matters: Clear docstrings help reviewers understand intent and help LLMaJ evaluate coverage.

How to fix:

PYTHON

 Copy

```
# Bad
def test_output():
    ...

# Good
def test_output_file_has_correct_format():
    """Verify output.json contains 'status' and 'items' fields."""
    ...
```

anti_cheating_measures

What it checks: Whether it's difficult for agents to cheat on the task.

Why it matters: Agents might try to:

- Look inside test files for answers
- Edit data files to pass tests
- Delete or modify tests
- Find solutions in git history

How to fix:

- Don't expose test logic to the task environment
- Use checksums or external validation
- If cloning repos, pin to specific commits
- Make tests verify actual computation, not just outputs

structured_data_schema

What it checks: If the agent produces structured data (JSON, CSV, etc.), is the exact schema described?

Why it matters: Ambiguous schemas lead to false failures.

How to fix:

MARKDOWN

 Copy

Output JSON with this structure:

```
```json
{
 "status": "success" | "error",
 "count": <integer>,
 "items": [
 {"id": <int>, "name": <string>}
]
}
```

 Copy

----

### hardcoded\_solution

**\*\*What it checks:\*\*** Whether the solution demonstrates a command seq

**\*\*Why it matters:\*\*** Solutions should show HOW to solve the task, no

**\*\*How to fix:\*\***

```
```bash
```

```
# Bad: Just outputs the answer
```

```
echo "42" > /output/result.txt
```

```
# Good: Shows the derivation
```

```
cd /app
```

```
python calculate.py input.txt > /output/result.txt
```

file_reference_mentioned

What it checks: If tests check for specific files, are those filenames mentioned in `instruction.md`?

Why it matters: Agents can't know to create files they weren't told about.

How to fix:

MARKDOWN

Copy

Save your results to `/output/analysis.json`

Quick Reference Table

CHECK	WHAT IT VALIDATES	FIX STRATEGY
behavior_in_task_description	Tests match instructions	Add requirements to
behavior_in_tests	Instructions have tests	Add tests for require
informative_test_docstrings	Tests have docstrings	Add docstrings to a
anti_cheating_measures	Hard to cheat	Remove cheating op
structured_data_schema	Schema is explicit	Define exact data fo
hardcoded_solution	Solution shows process	Derive answers, dor
file_reference_mentioned	Output files named	Mention all output fi

Debugging LLMaJ Failures

When LLMaJ checks fail:

- 1. **Read the feedback** — LLMaJ explains why it failed
- 2. **Review the specific issue** — Which test/requirement is problematic?

3. **Make targeted fixes** — Don't rewrite everything
4. **Re-run checks** — Verify the fix worked

Next Steps

- [CI Feedback Training](#)
- [CI Checks Reference](#)
- [Submit your task](#)

PREVIOUS

[CI Checks Reference](#)

NEXT

[Agent Review Reference](#)